A SQL query is not a KPI

Why a parameterised query is the wrong place to define a business metric, and what to use instead.

By **Raphael MANSUY**

A common pattern stores each engagement KPI as a SQL query with parameters: a date range, some filters and a slot for the grouping. That is enough for one number in one table. It is not enough when the business asks for the same KPIs by typology, tier and campaign, across years and markets, with subtotals and grand totals. **A query records how to produce an answer. It does not record what the answer means**, so the answer cannot be safely regrouped.

● THE PROBLEM

Totals you cannot trust, layouts you cannot change

Subtotals of rates and of unique audiences come out wrong when they are built from the figures on screen. Every new layout needs new SQL.

● THE CAUSE

The query keeps the result and drops the ingredients

It returns finished numbers at one grouping. What is needed to regroup them is gone: the numerator, the denominator, and who was counted.

● THE SOLUTION

Define each KPI once, with no grouping inside

Store the ingredients and the formula. Let the reader choose rows, columns and totals. Compute every cell and every total from its own data.

What it costs, on a small example

DELIVERY RATE BY CAMPAIGN · SYNTHETIC

CAMPAIGN	SENT	DELIVERED	DELIVERY RATE
Campaign A	90	81	90.0%
Campaign B	10	5	50.0%
Total by averaging the two rates	—	—	~~70.0%~~
Total over all messages	**100**	**86**	**86.0%**

Synthetic data. The correct total divides all delivered messages (86) by all sent messages (100). The 13-message data set of `examples/registry/` shows the error in the other direction: a delivery rate of 76.9%, against 81.9% for the average of its three typology cells.

−16 points

Delivery rate understated when the total averages the two campaigns.

9 not 6

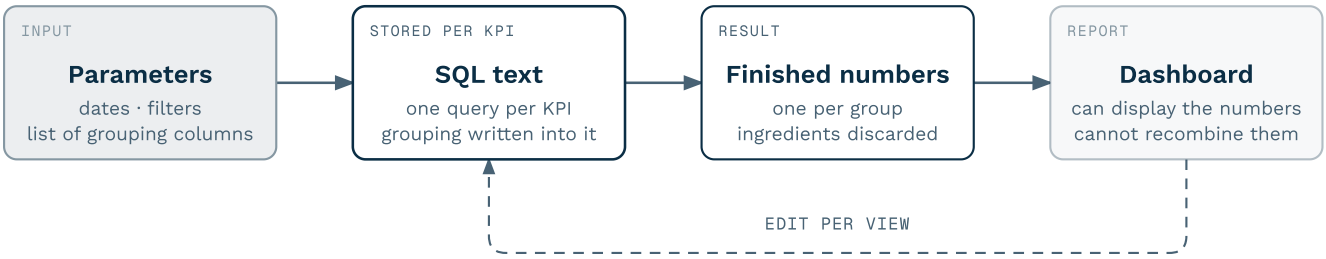
Clients reached when two market cells are added, against 6 distinct clients.

Why the average misleads. Campaign A carries 90% of the messages and delivers 90% of them. The average gives campaign B, with ten messages, half of the vote.

> *A query answers one question. A KPI has to answer the same question at every level of the report.*

Five reasons a parameterised query falls short

A parameterised query is SQL text with blanks: values for dates and filters, and a list of columns to group by. Filling the blanks runs the query and returns one finished number per group. The weaknesses below come from that design, not from anyone's SQL.



Today. The KPI lives in the SQL text. The report receives numbers it can show but cannot total, and a new layout sends someone back to edit the SQL.

1

It stores an answer, not a meaning

The query returns 90%. The 81 delivered and 90 sent messages behind it are not returned, so it cannot be combined with another figure.

BUSINESS IMPACT

Rates of 90% and 50% on 90 and 10 messages average to 70%; the answer is 86%.

2

The grouping is written into the text

A query computes at one level of detail. Another level means editing the text or pasting a new column list into it, which is text substitution and not a safe value parameter.

BUSINESS IMPACT

Each new layout is a change request, with a fresh chance of error each time.

3

It cannot say which kind of number it returns

Messages sent add up. A rate must be recomputed. Unique clients must be recounted, because one person can be in two campaigns. All three look the same coming out of a query.

BUSINESS IMPACT

The report guesses the total rule: 6 distinct clients were reached, yet two market cells add up to 9.

4

Every KPI works alone

Six KPIs at twelve combinations of detail, subtotal and grand total would be 72 separate executions in a straightforward design, although they read the same messages and share ingredients.

BUSINESS IMPACT

Run time and warehouse cost grow with every KPI and every subtotal added.

5

The business definition stays hidden

Owner, population, time basis and what counts as "reached" exist only inside SQL. Does Reach count messages or people? The query gives one answer and does not say which.

BUSINESS IMPACT

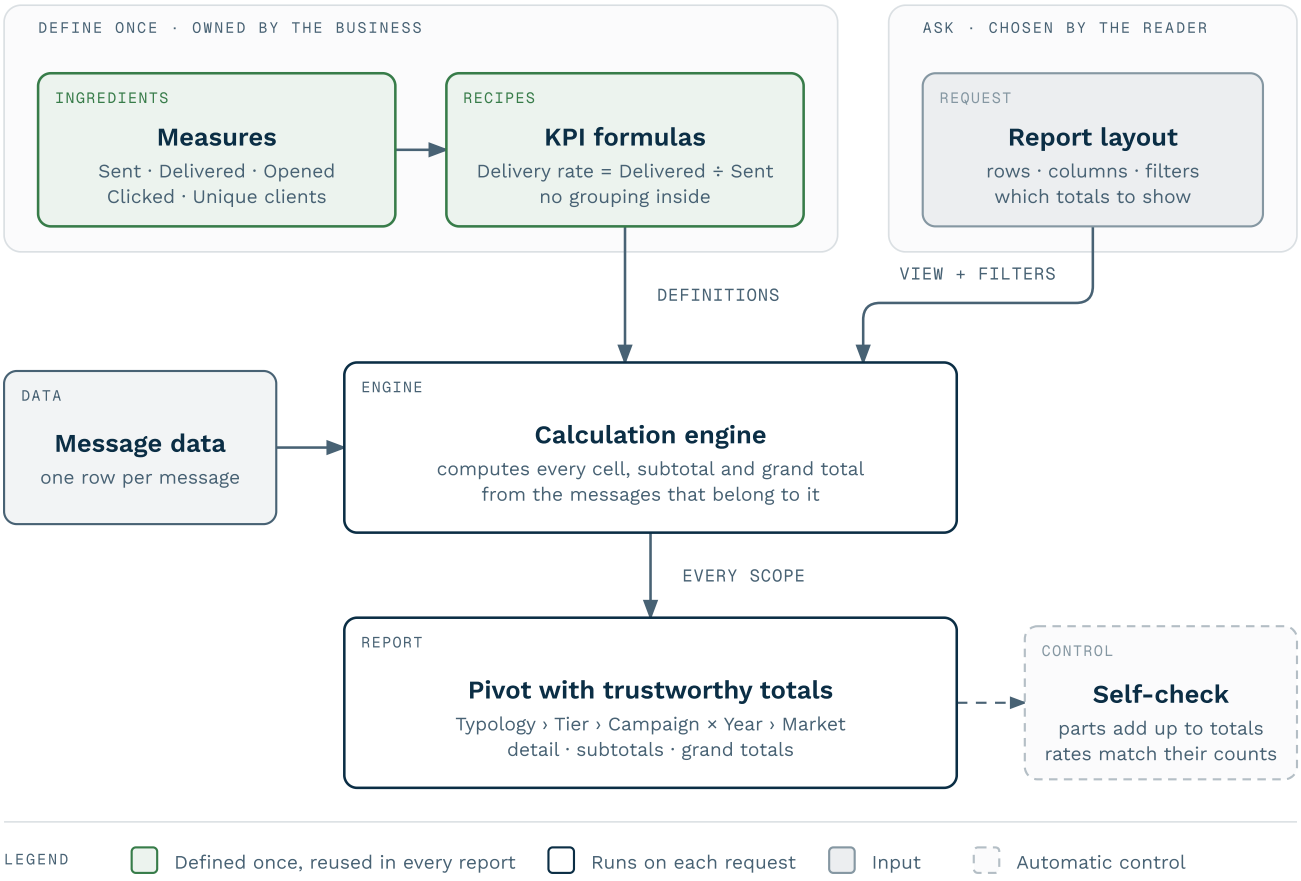
Owners cannot approve a definition they cannot read, and two reports can disagree unnoticed.

What parameterised queries do well

They bind dates and filter values safely, and they are a sound way to **run** a calculation. For a fixed report with one number per row they are adequate. The limits appear when the same KPI must be read at several levels at once. SQL stays the engine; it should stop being the definition.

Define the KPI once, then ask for any view

Separate what the business owns from what the reader chooses. The business defines **measures** (what is counted) and **formulas** (how measures combine into a KPI), with no grouping in either. The reader chooses the layout. An engine computes every cell and total from its own messages.



Target. The definition never changes when the layout does. A grand total uses the same formula as a detail cell.

	KPI AS A PARAMETERISED QUERY	KPI AS A DEFINITION
What is stored	SQL text returning one number per group	Measures and a formula, readable by the owner
Changing the layout	Edit or regenerate the SQL	Change the request; the definition is untouched
Subtotals and totals	Extra queries, or sums of shown figures	Each computed from its own data, same formula
Adding a KPI	A new query and a new pass over the data	Often one new formula over existing measures
Checking a result	Rerun and compare by hand	Automatic on every report

One formula, three scopes

DELIVERY RATE = DELIVERED ÷ SENT · SYNTHETIC EXAMPLE, 100 MESSAGES

DETAIL CELL

81 ÷ 90 = 90.0%

Campaign A

DETAIL CELL

5 ÷ 10 = 50.0%

Campaign B

GRAND TOTAL

86 ÷ 100 = 86.0%

Averaging the two campaign rates gives 70.0%

The definition, and three ways to build it

Three kinds of number, three rules for a total

STATED IN THE DEFINITION, NEVER GUESSED

KIND	EXAMPLE	RULE FOR A TOTAL	IF CELLS ARE SIMPLY COMBINED
Count	Messages sent	Add the parts, or count again	Correct, provided each message sits in one part only
Rate	Delivery, open and click rates	Total numerator ÷ total denominator	An average is right only when group size and group rate are unrelated
Unique audience	Clients reached	Count each person once across the whole scope	A sum counts a client again in each campaign, market or year

Three ways to implement it

EACH REPRODUCED THE SAME 56 TEST VALUES

PREFERRED

Databricks metric views

Measures are defined in the data catalogue and governed with the data. Native to Databricks, and tested on a Databricks warehouse.

ALTERNATIVE

Malloy

An open-source language built on this idea. It matched the same values in a local test. Consider it if several database engines must share one set of definitions.

FOR GAPS

Generated SQL

Definitions kept in a catalogue and compiled to SQL. The most control and the most to maintain; use it where the native option falls short.

In all three, SQL still executes the calculation. Pages 5 to 7 show each option on one example; page 8 compares them.

The example used on the next three pages

EIGHT MESSAGES · TYPOLOGY BY MARKET · WITH TOTALS

Each option answers the same small request: three KPIs, **Typology in rows, Market in columns, totals on both axes**. The data is eight messages, so every answer can be checked by hand.

The three KPIs cover the three kinds of number: Sent is a count, Delivery rate is a rate, Unique clients is a unique audience.

Look at Unique clients. Each market total shows 3 and the grand total shows 5, not 6: one VIP client received messages in both markets and counts once.

	HONG KONG	SINGAPORE	ALL MARKETS
New	3 · 66.7% · 2	1 · 100% · 1	**4 · 75.0% · 3**
VIP	1 · 100% · 1	3 · 66.7% · 2	**4 · 75.0% · 2**
All typologies	**4 · 75.0% · 3**	**4 · 75.0% · 3**	**8 · 75.0% · 5**

Each cell reads: messages sent · delivery rate · unique clients. Shaded cells are totals, each computed from its own messages. Synthetic data.

MESSAGE	CLIENT	TYPOLOGY	MARKET	DELIVERED
1	c1	New	Hong Kong	yes
2	c1	New	Hong Kong	yes
3	c2	New	Hong Kong	no
4	c3	New	Singapore	yes
5	c4	VIP	Hong Kong	yes
6	c4	VIP	Singapore	yes
7	c5	VIP	Singapore	yes
8	c5	VIP	Singapore	no

The eight messages. Client c4 appears in both markets, which is why unique clients cannot be added across markets.

Databricks metric views PREFERRED

A metric view is an object in the data catalogue that stores measures and the formulas built on them. Reports read it with ordinary SQL plus one function, `MEASURE()`, and choose the grouping in the query. It is preferred where the platform is Databricks, because the definition is governed alongside the data.

1 Define once

NO GROUPING IN THE DEFINITION

```
CREATE OR REPLACE VIEW crm.gold.engagement_kpi
WITH METRICS LANGUAGE YAML AS $$
version: 1.1
comment: "Engagement KPIs, one row per message sent"
source: crm.silver.message
dimensions:
  - name: typology
    expr: typology
  - name: market
    expr: market
measures:
  - name: sent
    expr: COUNT(*)
  - name: delivered
    expr: COUNT(*) FILTER (WHERE is_delivered)
  - name: unique_clients
    expr: COUNT(DISTINCT client_id)
  - name: delivery_rate
    expr: MEASURE(delivered) * 1.0 / NULLIF(MEASURE(sent), 0)
    comment: "Delivered messages divided by sent messages"
$$;
```

2 Ask for a view

READER'S CHOICE

```
SELECT typology, market,
       GROUPING_ID(typology, market) AS gid,
       MEASURE(sent)           AS sent,
       MEASURE(delivery_rate)  AS delivery_rate,
       MEASURE(unique_clients) AS unique_clients
FROM crm.gold.engagement_kpi
GROUP BY GROUPING SETS (
  (typology, market),   -- detail cells
  (typology),           -- typology totals
  (market),             -- market totals
  ()                    -- grand total
);
```

3 Read the result

ONE ROW PER GROUP

GID	THE ROW IS	ROWS HERE
0	a detail cell	4
1	a typology total, all markets	2
2	a market total, all typologies	2
3	the grand total	1

How it works

1. The view lists what may be grouped by (dimensions) and what is counted (measures). It contains no `GROUP BY`.
2. A rate is written from other measures. The engine evaluates numerator and denominator at whatever grouping the reader asks for, then divides.
3. `GROUPING SETS` requests detail, subtotals and grand total together. `gid` tells a total from a genuine "unknown" value.

Strengths

- One definition in Unity Catalog, with the permissions and lineage of the data.
- Consumers keep writing SQL. Nothing extra to run.
- The `comment` puts the business meaning next to the formula.

Watch for

- Write the `NULLIF` guard in every rate. In its default mode Databricks raises an error on division by zero.
- Exact unique counts at many levels can be slow. Measure before committing.
- Needs a recent runtime: 16.4 or later to create a metric view, per Databricks documentation.

TESTED

A version with five dimensions and twelve grouping sets ran on a Databricks serverless SQL warehouse on 12 November 2026 and matched all 56 groups. The two-dimension form on this page is that statement reduced; it has not been run in this form.

STILL TO PROVE BEFORE PRODUCTION

Report filters, row-level access rules under the reader's identity, empty and unknown groups, and cost at real volumes. These are the remaining acceptance tests.

Malloy

ALTERNATIVE

Malloy is an open-source language for describing data. A **source** declares measures once. A **query** nests one level of the report inside another. Malloy compiles the whole request into a single SQL statement and returns a tree shaped like the report.

1 Define once

NO GROUPING IN THE DEFINITION

```
source: message is duckdb.table('message') extend {
  measure:
    sent is count()
    delivered is count() { where: is_delivered }
    unique_clients is count(client_id)
    delivery_rate is delivered / nullif(sent, 0)
}
```

2 Ask for a view

ONE NEST PER REPORT LEVEL

```
query: pivot is message -> {
  // grand total
  aggregate: sent, delivery_rate, unique_clients

  // market totals
  nest: by_market is {
    group_by: market
    aggregate: sent, delivery_rate, unique_clients
  }

  // typology totals, with their detail cells inside
  nest: by_typology is {
    group_by: typology
    aggregate: sent, delivery_rate, unique_clients
    nest: by_market is {
      group_by: market
      aggregate: sent, delivery_rate, unique_clients
    }
  }
}
```

3 Read the result

A TREE, NOT FLAT ROWS

LEVEL RETURNED	SENT	RATE	CLIENTS
Grand total	**8**	**75.0%**	**5**
by_market · HK	4	75.0%	3
by_market · SG	4	75.0%	3
by_typology · New	**4**	**75.0%**	**3**
by_market · HK	3	66.7%	2
by_market · SG	1	100.0%	1
by_typology · VIP	**4**	**75.0%**	**2**
by_market · HK	1	100.0%	1
by_market · SG	3	66.7%	2

How it works

1. `measure:` names a calculation with no grouping. A rate is written from two other measures.
2. `group_by` with `aggregate` evaluates the measures at one level. `nest:` puts a finer level inside it.
3. Each level is computed from its own rows. Malloy writes one SQL statement that repeats each row once per level; it does not use `GROUPING SETS`.

Strengths

- The same model runs on several engines: connectors exist for DuckDB, BigQuery, Snowflake, Postgres, Trino and Databricks.
- The result has the shape of the report's row hierarchy.
- Open source, and the model is a text file under version control.

Watch for

- No automatic zero guard: an unguarded rate returned `NaN`. Write `nullif` in every rate.
- A column axis means repeating a nest at each row level, and the pivot code must flatten the tree.
- A second language and a runtime to operate. Packages are at version 0.0.x.

TESTED

This model and query ran on DuckDB with Malloy 0.0.437 and returned the nine groups above. The five-dimension version matched all 56 groups, also on DuckDB. Nothing was run on Databricks.

CHOOSE IT WHEN

The same definitions must run on more than one database engine, or a nested result is what the consumer needs. Then run the same acceptance tests through its Databricks connector first.

Generated SQL FOR GAPS

Keep the definitions in your own catalogue and let a small **planner** write the SQL for each request. It gives the most control and the most to maintain, so use it where the native option falls short.

1 Define once

CATALOGUE ENTRY · PROPOSED FORMAT

```yaml
measures:
  sent:
    expression: COUNT(*)
    kind: count
  delivered:
    expression: COUNT(*) FILTER (WHERE is_delivered)
    kind: count
  unique_clients:
    expression: COUNT(DISTINCT client_id)
    kind: unique

kpis:
  delivery_rate:
    label: Delivery Rate
    formula: ratio(delivered, sent)
    owner: CRM metric owner
    status: unconfirmed
```

2 Ask for a view

A REQUEST WITH NO SQL IN IT

```json
{
  "kpis": ["sent", "delivery_rate", "unique_clients"],
  "rows": ["typology"],
  "cols": ["market"],
  "totals": { "rows": true, "cols": true }
}
```

3 What the planner generates

PRODUCED BY THE PROTOTYPE PLANNER · EXECUTED

```sql
WITH agg AS (
  SELECT typology, market,
    GROUPING_ID(typology, market) AS gid,
    COUNT(*) AS sent,
    COUNT(*) FILTER (WHERE is_delivered) AS delivered,
    COUNT(DISTINCT client_id) AS unique_clients
  FROM message
  WHERE TRUE
  GROUP BY GROUPING SETS ((typology, market), (typology), (market), ())
  HAVING COUNT(*) > 0
)
SELECT typology, market, gid, sent, delivered, unique_clients, delivered * 1.0 / NULLIF(sent, 0) AS delivery_rate
FROM agg
```

4 Read the result

NINE ROWS, THREE SHOWN

typology	market	gid	sent	delivery_rate	unique_clients
New	HK	0	3	0.667	2
VIP	null	1	4	0.750	2
null	**null**	**3**	**8**	**0.750**	**5**

How it works

1. The planner checks every requested name against the catalogue and refuses unknown ones.
2. It writes each needed measure once in the inner block, with one grouping set per report level.
3. Formulas run outside, after aggregation. `ratio()` always adds the zero guard, and `HAVING` drops empty groups.

Strengths

- Full control of the plan: a heavy unique count can run in its own statement.
- Works on any SQL engine.
- Guards, validation and result checks live in one place.

Watch for

- You own the planner: validation, security, totals and tests.
- Names come from the catalogue and values are bound as parameters, never pasted into text.
- Avoid two definitions of one KPI if metric views exist too.

TESTED

Generated by the prototype planner and run on DuckDB: the nine expected groups. The five-dimension version matched all 56 groups on DuckDB and on a Databricks warehouse. The catalogue and request formats are proposals.

CHOOSE IT WHEN

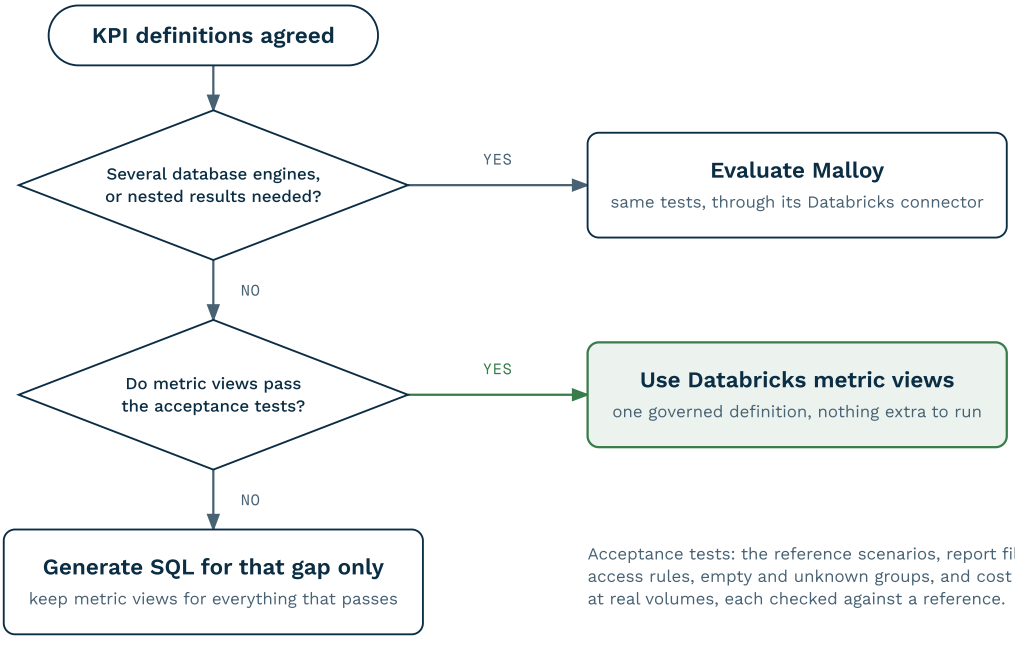
An acceptance test shows a gap in the native option, such as a total it cannot express or a unique count that is too slow. Build that capability only.

Choosing between the three

All three keep the KPI definition free of grouping and compute each total from its own data, so they return the same numbers. They differ in where the definition lives, what has to be operated, and how much has been proven on Databricks. Whichever is chosen, each KPI keeps one authoritative definition.

	DATABRICKS METRIC VIEWS	MALLOY	GENERATED SQL
Where the definition lives	An object in Unity Catalog	A model file under version control	Your own catalogue tables
How a report asks	SQL with `MEASURE()` and grouping sets	A Malloy query with nested levels	A request: KPIs, rows, columns, totals
What comes back	Flat rows with a grouping id	A tree shaped like the report	Flat rows with a grouping id
Zero-denominator guard	Written in each rate	Written in each rate	Generated for every rate
Access control	Unity Catalog; to confirm in acceptance tests	Through the database connection; untested	To build: run as the reader
Extra software to run	None	Malloy compiler and runtime	The planner service
Proven so far	56 of 56 groups on a Databricks warehouse	56 of 56 groups on local DuckDB	56 of 56 groups on DuckDB and a Databricks warehouse
Choose it when	Databricks is the platform	Several engines, or nested output	The native option has a proven gap

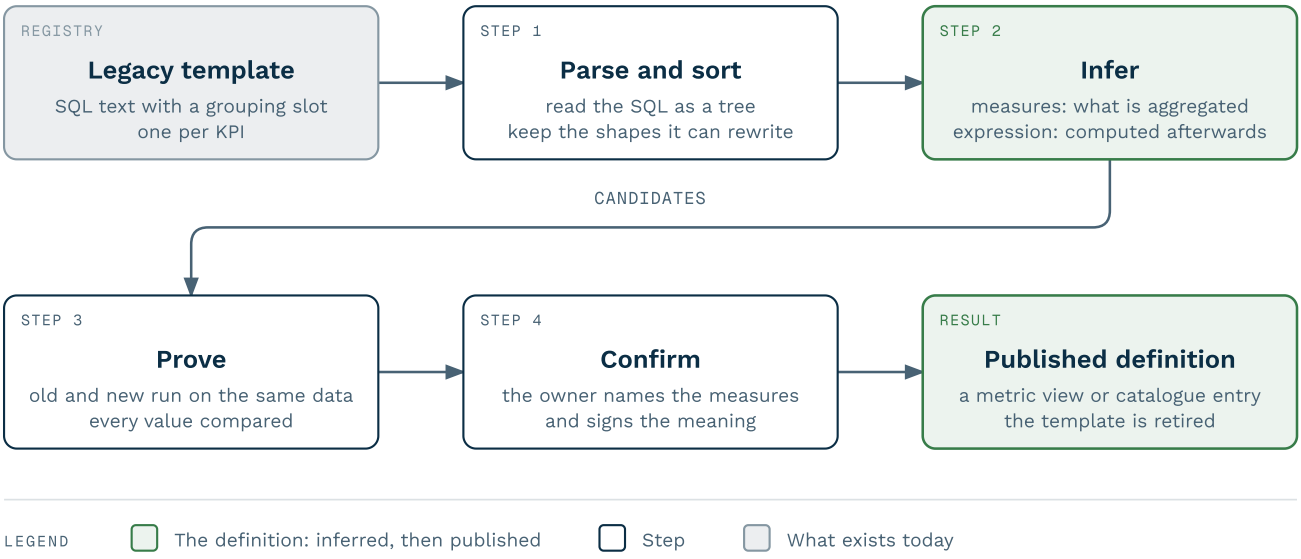
A decision path START ONCE THE KPI DEFINITIONS ARE AGREED



Preferred path in green. Diamonds are decisions, rectangles are outcomes.

Migrate by inferring the definitions

A registry table typically holds every KPI as a SQL template with a grouping placeholder such as `{{ group_by_list }}`. Each template already contains the **measures** and the **expression** of its KPI; they are simply not separated. A tool can read the template, infer both, and prove that the result gives the same numbers. A person then names the measures and confirms the meaning.



The path of one KPI. Steps 1 to 3 are automatic. Step 4 is a person's decision and cannot be automated.

What can be inferred, and what a person decides

THE LINE BETWEEN THE TOOL AND THE OWNER

INFERRED FROM THE SQL	DECIDED BY A PERSON
The source table and the row filters	Whether that population is the intended one
Each aggregate, as a measure	A business name for each measure
The expression that combines the measures	Whether it matches what the KPI should mean (Reach, click-to-open rate)
Measures that several KPIs share	Which definition stands when two KPIs disagree
Whether old and new return the same values	Whether a difference is a defect or a behaviour to keep

Three outcomes for a template

TWELVE TEST TEMPLATES: NINE REWRITTEN, THREE REFUSED

AUTOMATIC

Rewritten as it stands

One aggregation over one source, grouped by the slot alone, with its parameters and optional filters kept as they are.

WITH A DECLARED RULE

Rewritten after a check

A join is accepted when the data shows it keeps one row per message. A row filter used by only some KPIs moves into the measure, which raises the empty-group question.

BY HAND

Listed and left alone

Nested aggregations, window functions, joins that multiply rows, several sources. The tool refuses them and says why.

The inference, on three templates

Three templates go in. Three measures, three expressions and one statement with every total come out. The legacy templates, run one grouping at a time, are the reference.

1 Before

TEMPLATES IN THE REGISTRY STYLE

```
-- kpi: sent
SELECT {{ group_by_list }}, COUNT(*) AS kpi_value
FROM message C
WHERE 1 = 1
GROUP BY {{ group_by_list }};

-- kpi: delivery_rate
SELECT {{ group_by_list }},
       COUNT(*) FILTER (WHERE C.is_delivered) * 1.0
         / NULLIF(COUNT(*), 0) AS kpi_value
FROM message C
WHERE 1 = 1
GROUP BY {{ group_by_list }};

-- kpi: reached_clients
SELECT {{ group_by_list }},
       COUNT(DISTINCT C.client_id) AS kpi_value
FROM message C
WHERE 1 = 1 AND C.is_delivered
GROUP BY {{ group_by_list }};
```

2 Inferred

OUTPUT OF THE TOOL

MEASURES · THREE FOUND, ONE SHARED

m1	`COUNT(*)`	Used by Sent and by Delivery rate
m2	`COUNT(*) FILTER(WHERE C.is_delivered)`	Used by Delivery rate
m3	`COUNT(DISTINCT C.client_id) FILTER(WHERE C.is_delivered)`	Used by Reached clients. Its row filter moved into the measure

EXPRESSIONS · APPLIED AFTER AGGREGATION

`sent = m1`
`delivery_rate = m2 * 1.0 / NULLIF(m1, 0)`
`reached_clients = m3`

One source, with no join. A fourth template, messages per client, was refused: it has more than one aggregation.

3 After

ONE STATEMENT FOR THE THREE KPIS · GENERATED AND EXECUTED

```
WITH agg AS (
  SELECT typology, market,
    GROUPING_ID(typology, market) AS gid,
    COUNT(*) AS m1,
    COUNT(*) FILTER(WHERE C.is_delivered) AS m2,
    COUNT(DISTINCT C.client_id) FILTER(WHERE C.is_delivered) AS m3
  FROM message AS C
  WHERE TRUE
  GROUP BY GROUPING SETS ((typology, market), (typology), (market), ())
  HAVING COUNT(*) > 0
)
SELECT typology, market, gid,
  m1 AS sent,
  m2 * 1.0 / NULLIF(m1, 0) AS delivery_rate,
  m3 AS reached_clients
FROM agg
```

The three rewrites

1. The grouping slot becomes grouping sets: each total is computed from the rows.
2. Each aggregate becomes a measure; the rest is the expression. `COUNT(*)`, used by two KPIs, is kept once.
3. A row filter not shared by every KPI moves into its measure.

What the test showed

- 31 legacy values compared, at detail and at every total. None differ.
- The legacy template cannot express a grand total: an empty column list is a syntax error.
- The nested template was refused, with a reason.

Watch for

- Empty groups: a market with nothing delivered shows 0 reached clients after migration, and no row before. Decide which is right.
- `m1` to `m3` are placeholders for the owner to name.

TESTED

Four templates, nine synthetic messages, DuckDB, sqlglot 30.25.0.

CONTINUED ON THE NEXT PAGE

Joins, date parameters, optional filters, Databricks functions and six common engagement KPIs.

Joins, parameters and Databricks functions

A second test uses six common engagement KPIs (message-sent, message-reached, delivery-rate, open-rate, click-rate, click-to-open-rate), written with a dimension join, optional date parameters and Databricks functions. It ran on DuckDB and natively on a Databricks warehouse.

1 One of the six templates

OPEN RATE · REGISTRY-STYLE TEMPLATE

```
-- kpi: open-rate | open_rate
SELECT {{ group_by_list }},
       try_divide(COUNT_IF(C.is_opened), COUNT_IF(C.is_delivered)) AS kpi_value
FROM message C
LEFT JOIN client D ON D.client_id = C.client_id
WHERE 1 = 1
  AND (try_cast(:start_date AS DATE) IS NULL
       OR CAST(C.send_date AS DATE) >= try_cast(:start_date AS DATE))
  AND (try_cast(:end_date AS DATE) IS NULL
       OR CAST(C.send_date AS DATE) < try_cast(:end_date AS DATE))
GROUP BY {{ group_by_list }};
```

2 Inferred measures

FIVE BEHIND SIX KPIS

m1	`COUNT(*)`	Sent, Delivery rate
m2	`COUNT(DISTINCT C.client_id) FILTER(WHERE C.is_delivered)`	Reached
m3	`COUNT(*) FILTER(WHERE C.is_delivered)`	Delivery rate, Open rate, Click rate
m4	`COUNT(*) FILTER(WHERE C.is_opened)`	Open rate, CTOR
m5	`COUNT(*) FILTER(WHERE C.is_clicked)`	Click rate, CTOR

3 Inferred expressions

AND WHAT ELSE WAS READ

`sent = m1`
`reached = m2`
`delivery_rate = TRY_DIVIDE(m3, m1)`
`open_rate = TRY_DIVIDE(m4, m3)`
`click_rate = m5 * 1.0 / NULLIF(m3, 0)`
`ctor = TRY_DIVIDE(m5, m4)`
Source. Messages joined to clients, one row per message.
Filters. Two optional dates, `:start_date` and `:end_date`.

The five cases

WHAT THE TOOL DOES, AND WHAT THE TEST SHOWED

CASE	WHAT THE TOOL DOES	RESULT OF THE TEST
Joins	Runs the join on the data and accepts it only if there is still one row per message	Client join kept. Event join refused: 13 messages become 14 rows
Date parameters	Keeps `:start_date` and `:end_date` as named parameters, never as pasted text	Bound natively on Databricks, empty values included
Optional filters	Recognises the `:p IS NULL OR …` pattern and keeps it as a row filter shared by all KPIs	Same values with no date, with a start date, and with both dates
Databricks functions	Reads `try_cast`, `try_divide` and `count_if`, and writes Databricks SQL back	Legacy templates and migrated statement both ran on Databricks
Six common KPIs	Finds five measures behind six KPIs; three spellings of a conditional count become one	200 legacy values compared. None differ

TESTED

DuckDB and native Databricks, 93 statements, 12 November 2026. Three parameter settings, 34 groups, 200 legacy values, 18 of them empty on both sides for a zero denominator.

STILL OPEN

A production registry. These six templates are written for the test, and a real one will hold shapes not covered here. Also dimensions that change over time, and several fact tables.

Migration in waves, with proof at each step

Inference makes the first pass over a whole registry cheap, so the plan starts with an inventory and leaves the difficult KPIs for last. Each wave ends with evidence that a business owner can read.

Five waves

EACH ENDS WITH EVIDENCE, NOT A DATE

1	**Inventory**	Run the inference over every registry template. Each KPI is sorted into automatic, rule-based or by hand, and shared measures are listed. This sizes the rest of the work.
2	**Pilot**	Message Sent and Delivery Rate. Old and new agree at every detail group, totals match an independent calculation, and the owner signs the two definitions.
3	**Bulk**	The remaining automatic and rule-based templates, in batches by source table. One comparison report per KPI, with every difference explained.
4	**By hand**	Nested and multi-source KPIs, one at a time, each with a written definition and its own tests.
5	**Retire**	Reports switch to the published definitions. Old and new run side by side for an agreed period. A template is removed only after its owner signs off.

Safeguards

WHAT KEEPS A MIGRATED KPI TRUSTWORTHY

Detail first. Compare old and new at detail level, where the legacy value is the reference.

Totals against an independent calculation. The legacy path cannot produce them, so it cannot be their reference.

Same data on both sides. One data version, the same filters and the same access rights for every comparison.

One rule for empty groups, decided once and applied to every KPI.

The template stays until sign-off. Rolling back means pointing the report at it again.

One definition per KPI. After sign-off the template is retired, so two sources cannot drift apart.

What inference cannot do

It reads what the SQL does, not what the KPI should mean. A join that counts a message twice is carried over faithfully, and Reach or click-to-open rate are no clearer after parsing than before. Inference shortens the migration. The owner's decision is still what closes it.

Who does what

PROPOSED ROLES

Runs the inference and the comparisons, and maintains the rewrite rules	**Engineering**
Checks that each source has one row per message and that joins keep it so	**Data owner**
Names the measures, confirms the meaning, accepts or rejects each difference	**KPI owner**

What the business decides, and the order of work

Decisions the business owns NO TOOL CAN MAKE THESE

- **Reach.** Messages or people? The readings give different totals: on the 13 synthetic messages, 10 were delivered to 6 distinct clients. A quick test: compare a quarter with the sum of its months. Equal suggests messages; lower means people.
- **Click-to-open rate.** Clicks ÷ opens, or clicks ÷ delivered? Confirm the name and the denominator.
- **Unknown values.** Whether a message from a client missing in the dimension appears as "unknown" or is dropped.
- **Exact or estimated audiences.** Exact by default. An estimate is faster and must be labelled as one.
- **Which totals to show**, for example market across all years, and whether a subtotal with a single line beneath it is displayed.

Order of work EACH STEP ENDS WITH EVIDENCE, NOT A DATE

1	**Agree the definitions and choose the option**	Each KPI written as owner, population, numerator and denominator, and signed. The same acceptance tests run on the candidate options.
2	**Build the smallest working core**	Sent and Delivery rate match an independent check at every detail cell and total.
3	**Move the confirmed KPIs across**	Measures and expressions are inferred from the registry templates (pages 9 to 12). Each migrated KPI is reconciled with the current one, and every difference is accepted by its owner.
4	**Release the pivot**	Access rules are enforced, the automatic self-check is on, and run time and cost meet an agreed budget.
5	**Add further capabilities one at a time**	Period comparisons, combined audiences, estimates and optimisations, each with its own definition and test.

Where the detail is IN THE SPECIFICATION REPOSITORY

The implementation examples and the inference prototype	`examples/walkthrough/`
The Databricks run of the migration test	`specs/kpi/validation/2026-11-12-migration/`
The registry migration test: six KPIs, 13 messages	`examples/registry/`
Why totals break, worked on eight messages	`specs/kpi/START-HERE.md`
The solution decision and the alternatives	`specs/kpi/05-first-principles-solution.md`
The rules for totals, with the algebra and the self-checks	`specs/kpi/16-correctness-contract.md`
What has been tested and what has not	`specs/kpi/14-test-strategy-and-traceability.md`

Keep SQL as the engine. Move the meaning of each KPI to a definition the business can read, own and reuse.

Evidence and limits. All examples use synthetic data: 100 messages on pages 1 and 3, eight on pages 4 to 7, thirteen in `examples/registry/`. The 72 executions are arithmetic. The metric-view example was not run in its reduced form. No production total, run time or cost has been measured.